

Lucrare practică nr. 1

Tema: Crearea unei rețele neuronale artificiale (RNA)

Scopul lucrării: Crearea unei rețele neuronale cu straturi Dense (Python) pentru clasificare binară.

Obiectivele lucrării:

1. Crearea unei rețele neuronale cu straturi Dense pentru operații simple în Python.
2. Antrenarea rețelei și validarea rezultatelor.
3. Optimizarea rețelei.

Numărul de ore necesare pentru realizare – 4 ore academice.

5.2 Noțiuni generale:

Exemplu de creare a unei Rețele Neuronale Artificiale în Python:

Setul de date (Churn_Modelling.csv) reprezintă detaliile clienților dintr-o companie [14]. Coloanele conțin informații despre salariul estimat, vârsta, sexul, etc. Scopul este să ofere toate detaliile despre un angajat.

Pe baza datelor angajaților unei bănci, se determină dacă un angajat are șansa de a rămâne în companie sau nu.

1. Importarea librăriilor și citirea datelor

Pasul 1: Importați `pandas`, `numpy`, `matplotlib` și `seaborn`. Apoi setați `%matplotlib inline` (veți importa apoi `sklearn` după necesitate).

Pasul 2: Citiți fișierul `Churn_Modelling.csv` ca `DataFrame` numit `data`.

Pasul 3: Citiți primele linii (`head`) despre angajați și aflați informații despre ei folosind funcțiile `info()` și `describe()`.

2. Analiza exploratorie a datelor

Pasul 4: Ștergeți coloanele care nu sunt importante: `drop(['RowNumber', 'CustomerId', 'Surname'])`

Pasul 5: Verificați dacă `data` are valori nule.

Pasul 6: Folosind `LabelEncoder()` transformați tipurile de date obiect în integer.

Pasul 7: Vizualizați datele (`head`) pentru observarea modificărilor efectuate.

3. Date de instruire și testare

Împărțirea datele în seturi de antrenament și de testare.

Pasul 8: Setați o variabilă `X` egală cu caracteristicile numerice ale angajaților și o variabilă `y` egală cu coloana `Exited`.

Pasul 9: Utilizați `model_selection.train_test_split` din `sklearn` pentru a împărți datele în seturi de antrenament și de testare. Setați `test_size=0,3` și `random_state=101`.

4. Crearea și antrenarea modelului modelului [15-16]

Pasul 10: Începeți crearea modelului cu `Sequential Object`. Apoi adăugați obiectul de intrare și specificați dimensiunea pe care doriți să o transmiteți. Adăugați neuronii din primul strat, din al 2-lea. Creați stratul final cu funcția `Sigmoid` pentru prezicere (0/1).

Pasul 11: Imprimați rezumatul `model.summary()` pentru a înțelege fluxul rețelei dvs. neuronale.

Pasul 12: Declarați optimizatorul și compilați modelul.

Pasul 13: Salvați valorile din timpul antrenării rețelei `history = model.fit`

5. Evaluarea modelului

Pasul 14: Evaluarea performanței modelului prin reprezentarea gra-fică a acurateței și a funcției loss [17].

Pasul 15: Importați metricile de evaluare a clasificării: `from sklearn.metrics import accuracy_score , confusion_matrix, classification_report`.

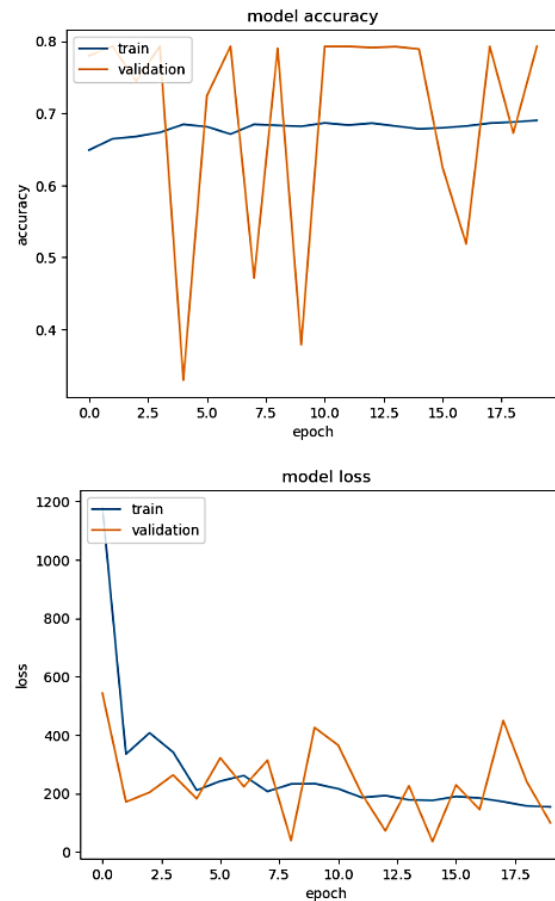


Figura 5.1 Reprezentarea grafică a funcțiilor loss și accuracy

Pasul 16: Imprimați rezultatele obținute:

```
# Check Accuracy
from sklearn.metrics import accuracy_score , confusion_matrix
accuracy_score(Ytest , predictions) # Accuracy Seems good
```

```
0.7926666666666666
```

```
confusion_matrix(Ytest , predictions)
```

```
array([[2377,   1],
       [ 621,   1]], dtype=int64)
```

```
from sklearn.metrics import classification_report
print(classification_report(Ytest , predictions))
```

	precision	recall	f1-score	support
0	0.78	0.70	0.74	2378
1	0.18	0.25	0.21	622
accuracy			0.61	3000
macro avg	0.48	0.47	0.47	3000
weighted avg	0.66	0.61	0.63	3000

Figura 5.2 Rezultatele clasificării cu RNA

6. Optimizarea RNA

Se observă că conform predicțiilor mai mulți angajați tind să rămână în companie decât să plece: 2378 vs. 622. Datele nu sunt echilibrate.

Pasul 17: Adăugați ponderi de clasă atunci când Datele sunt dezechilibrate:

```
from sklearn.utils.class_weight import compute_class_weight
```

Pasul 18: Antrenați modelul și observați rezultatele.

```
confusion_matrix(Ytest , predictions)
```

```
array([[1910,  468],
       [ 159,  463]], dtype=int64)
```

Figura 5.3 Rezultatele clasificării cu RNA după adăugarea ponderilor pentru clase

Prezicerea pentru angajații care vor să plece e mai bună, dar s-a înrăutățit pentru cei care rămân.

Pasul 19: Aplicați următoarele metode de optimizare:

- creșteți numărul de straturi;
- creșteți numărul de neuroni în straturi Dense;
- adăugați BatchNormalization la straturi;
- adăugați Dropout (abandon) după straturi.

Pasul 20: Vizualizați noul model creat.

Pasul 21: Declarați optimizatorul și compilați modelul.

Pasul 22: Salvați valorile din timpul antrenării rețelei `history = model.fit`

Pasul 23: Evaluați modelul.

```
# print summary to understand your neural network flow
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
batch_normalization_6 (Batch Normalization)	(None, 10)	40
dense_14 (Dense)	(None, 128)	1408
dropout_4 (Dropout)	(None, 128)	0
batch_normalization_7 (Batch Normalization)	(None, 128)	512
dense_15 (Dense)	(None, 64)	8256
dropout_5 (Dropout)	(None, 64)	0
batch_normalization_8 (Batch Normalization)	(None, 64)	256
dense_16 (Dense)	(None, 32)	2080
dense_17 (Dense)	(None, 1)	33
Total params: 12585 (49.16 KB)		
Trainable params: 12181 (47.58 KB)		
Non-trainable params: 404 (1.58 KB)		

Figura 5.4 Modelul RNA îmbunătățit

5.2 Sarcina individuală:

1. Consultați <https://www.kaggle.com/datasets> pentru a alege și a descărca un set de date pentru clasificare.
2. Importați librăriile și citiți fișierul ales.
3. Faceți analiza exploratorie a datelor (EDA), dacă este necesar curățați datele.
4. Parcurgeți pașii 1-23 ca în exemplu de mai sus
5. Scrieți concluzii.

5.3 Evaluare și livrabile

Studentii vor demonstra cunoștințele și abilitățile dobândite ca urmare a realizării lucrării practice. Vor explica pașii de creare a unei rețele, ce influențează alegerea numărului de neuroni, numărului de straturi, vor răspunde la întrebări referitoare la optimizatori, dropout, batch normalisation etc.

Studentii vor prezenta un fișier cu extensia .py și un raport .doc care conține: tema, obiectivele, sarcina, codul sursă și capturi de ecran cu rezultatele obținute la rularea codului și o concluzie despre ce au reușit să asimileze efectuând sarcinile din lucrare.

Modelul de raport poate fi consultat în Anexa 1.

Întrebări de autoevaluare:

1. Care sunt pașii de crearea a unei rețele neuronale artificiale (RNA)? Explicați modelul perceptronului.
2. Cum determinăm numărul de straturi de neuroni?
3. Cum alegem funcția de activare?
4. Ce calculează funcția loss?
5. Explicați gradientul de coborâre. Ce rată de învățare e recomandată?
6. Cum ne ajută optimizatorii?
7. Când și cu ce scop folosim straturile de abandon (Dropout Layers)? Ce rată de abandon e recomandată?
8. Care e scopul normalizării loturilor (Batch Normalization)?
9. Cum evaluăm rezultatele obținute cu ajutorul RNA?
10. Care sunt pașii de optimizare a unei rețele neuronale?